

Helix OTA — Version Roadmap

Document ID: HELOTA-ROADMAP-001
Version: 1.0.0-draft
Status: Active
Last Updated: 2026-03-04
Constitution Reference: HelixConstitution v1 §1-§4

Table of Contents

- 1. [Roadmap Philosophy](#)
 - 2. [Guiding Principles](#)
 - 3. [Version Overview Matrix](#)
 - 4. [1.0.0-MVP — Minimum Viable Product](#)
 - 5. [1.0.1-rollback — Rollback Support](#)
 - 6. [1.0.2-delta-updates — Delta/Differential Updates](#)
 - 7. [1.1.0-linux-support — Linux Distribution OTA](#)
 - 8. [1.2.0-windows-support — Windows OTA](#)
 - 9. [2.0.0-multi-os-universal — Universal Multi-OS](#)
 - 10. [Cross-Cutting Concerns](#)
 - 11. [Appendix A — Submodule Dependency Map](#)
 - 12. [Appendix B — Testing Standards Reference](#)
-

1. Roadmap Philosophy

The Helix OTA version roadmap is a **contract**, not a wishlist. Every version boundary is a hard gate: no feature crosses the gate without passing mutation testing, nano-detail documentation, and anti-bluff verification per the HelixConstitution.

Why Versioned Milestones?

An OTA system operates at the intersection of **safety-critical infrastructure** and **distributed systems correctness**. A failed update can brick a device, and a compromised update pipeline can compromise an entire fleet. We do not ship “mostly working” update mechanisms.

The roadmap follows a strict additive strategy:

MVP (Android-only) → Rollback → Delta → Linux → Windows → Universal

Each version **extends** the previous one without breaking backward compatibility. There are no “rewrite” versions—only additive capability layers.

Anti-Bluff Guarantee

Per HelixConstitution §2.3, the anti-bluff principle states:

No feature is “done” until it has been tested under mutation, documented to nano-detail, and verified against its acceptance criteria by an independent reviewer.

This means the roadmap timeline is **evidence-bound**, not date-bound. A version ships when its acceptance criteria are met, not when a calendar says so.

2. Guiding Principles

#	Principle	Source	Application
P1	Anti-Bluff	HelixConstitution §2.3	No feature declared complete without mutation-tested evidence
P2	Test-First	HelixConstitution §1.1	Every feature has tests written before implementation; mutation testing coverage ≥ 85%
P3	Nano-Detail Docs	HelixConstitution §3.1	Every API endpoint, data model, and state transition

#	Principle	Source	Application
P4	Fail-Safe by Default	HelixConstitution §4.2	documented to specification grade A/B partition design; updates never destroy the running system
P5	Minimal Trusted Compute Base	Architecture Decision	Only update_engine + our client touch partitions; no third-party OTA agents
P6	Submodule Reuse	vasic-digital Constitution	Leverage existing submodules; never re-implement solved problems
P7	Progressive Delivery	Industry Standard (Google Omaha)	Rollout 0→100% in configurable increments; pause/resume at any point

3. Version Overview Matrix

Version	Codename	Scope	Estimated Complexity	Key Deliverable
1.0.0-MVP	—	Android 15 / RK3588 OTA	Large (12–16 weeks)	Full A/B update pipeline for Orange Pi 5 Max
1.0.1-rollback	rollback	Multi-version rollback	Medium (4–6 weeks)	Safe rollback for last N images
1.0.2-delta-updates	delta	Differential payloads	Large (6–8 weeks)	60–90% bandwidth reduction
1.1.0-linux-support	linux	Ubuntu/Debian/Fedora/Arch	Large (8–12 weeks)	Linux distribution OTA

Version	Codename	Scope	Estimated Complexity	Key Deliverable
1.2.0-windows-support	windows	Windows Update + MSI/MSIX	Large (8-10 weeks)	Windows OTA client
2.0.0-multi-os-universal	universal	Plugin architecture	Extra Large (12-16 weeks)	OS-agnostic update pipeline

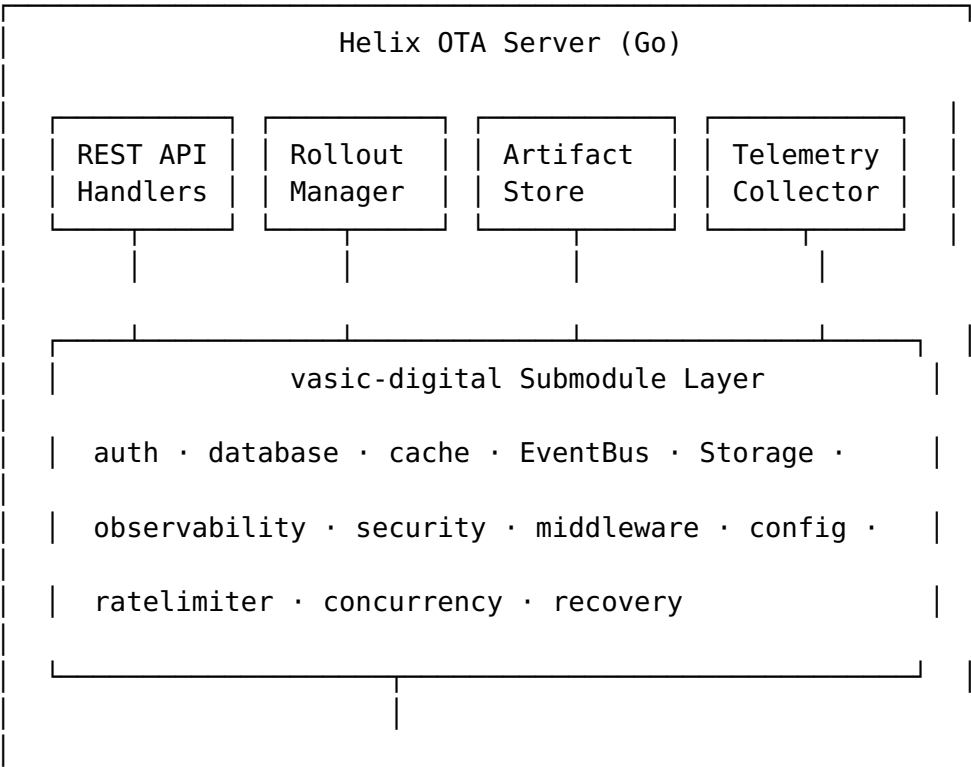
Note: Complexity estimates assume a team of 2-3 engineers working full-time. Actual timelines adjust based on mutation testing throughput (HelixConstitution §1.1).

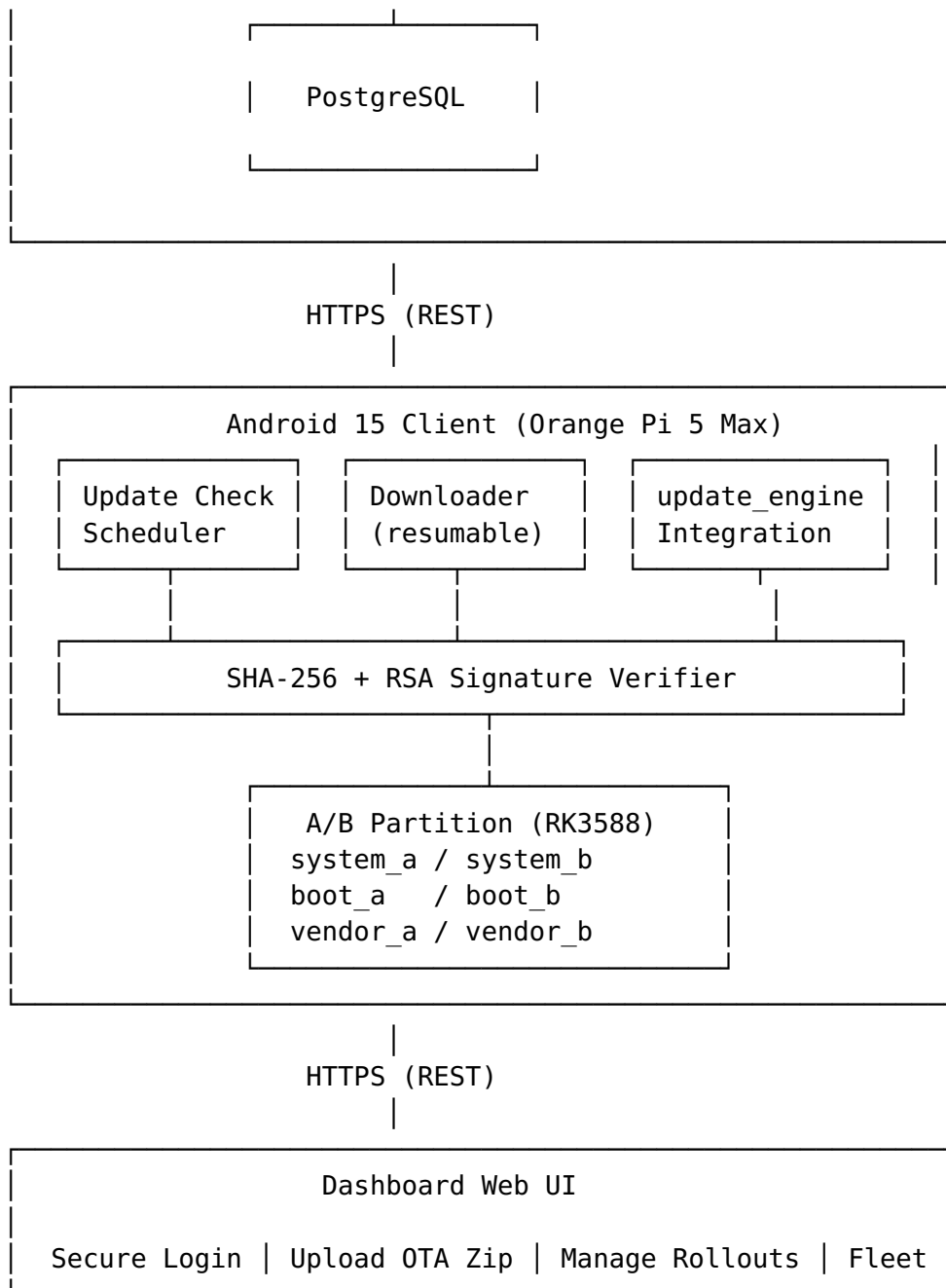
4. 1.0.0-MVP — Minimum Viable Product

4.1 Overview

The MVP delivers a complete, production-grade OTA update system for **Android 15 on Orange Pi 5 Max (RK3588)**. It covers the full lifecycle: device registration → update check → download → verify → apply → report.

4.2 Architecture





4.3 OTA Server (Go) — Feature List

Feature	Description	Acceptance Criteria
REST API — Update Check	GET /api/v1/devices/{id}/update-check returns available update or 204 No Update	Response < 200ms p95; returns payload URL, SHA-256, size, version when update available
REST API — Artifact Download	GET /api/v1/artifacts/{id}/download	Supports resume via Range header; returns

Feature	Description	Acceptance Criteria
REST API — Status Report	streams OTA zip with Range header support	Content-Length, SHA-256 in headers
	POST /api/v1/devices/{id}/status accepts device update status events	Accepts: DOWNLOADING, DOWNLOAD_PAUSED, VERIFYING, APPLYING, REBOOTING, SUCCESS, FAILED; idempotent
REST API — Artifact Upload	POST /api/v1/artifacts uploads OTA zip with validation	Validates: zip structure (payload.bin + payload_properties.txt + care_map.pb), SHA-256 match, RSA signature present; rejects invalid artifacts with 422
REST API — Rollout Management	POST /api/v1/rollouts creates rollout; PATCH /api/v1/rollouts/{id} updates percentage	Rollout: 0-100% in configurable increments (1%, 5%, 10%, 25%); pause/resume; target device groups
Device Registration	POST /api/v1/devices registers device with hardware fingerprint	Required fields: serial, model (rk3588_opi5max), current_version, slot_suffix; returns device ID + JWT
Device Authentication	Mutual TLS or JWT-based device auth	Devices authenticate on every request; invalid auth → 401; token rotation every 24h
Artifact Validation	Server-side validation of uploaded OTA zips	Reject: wrong payload format, missing signature, SHA-256 mismatch, unsupported target device; validation runs in < 30s for 2GB payload
Rollout Engine	Percentage-based rollout assignment	Deterministic device→cohort assignment (hash(device_id) mod 100 < rollout_pct);

Feature	Description	Acceptance Criteria
		cohort membership stable across check-ins

4.4 Android 15 Client — Feature List

Feature	Description	Acceptance Criteria
update_engine Integration	Drive update_engine via Binder IPC to apply A/B updates	Successfully calls ApplyPayload() with downloaded payload; handles onStatusUpdate callbacks; maps engine states to server status events
Periodic Update Checking	Check server for updates every N hours (configurable, default 4h)	Uses WorkManager for scheduling; respects device idle / battery constraints; exponential backoff on failure (1h → 2h → 4h → 8h → 24h max)
Resumable Download	Download OTA zip with HTTP Range resume	Handles network interruption gracefully; resumes from last byte; stores partial download in encrypted app-specific storage; cleans up on completion
SHA-256 + RSA Verification	Verify payload integrity and authenticity before applying	SHA-256 computed over payload.bin matches server-provided digest; RSA-2048/4096 signature verified against embedded public key; rejects tampered payloads
A/B Partition Update	Apply update via update_engine to inactive slot	Writes to inactive slot only; post-install verification via update_engine; sets active slot via IBootControl HAL; never modifies running slot

Feature	Description	Acceptance Criteria
Status Reporting	Report update lifecycle events to server	Reports at each state transition; includes error details on failure; retries on network error with backoff; offline queue with max 1000 events

4.5 Dashboard Web UI — Feature List

Feature	Description	Acceptance Criteria
Secure Login	Admin authentication with MFA support	Login via username/ password + TOTP; session timeout 30min; RBAC: admin, operator, viewer
Upload OTA Zip	Upload and validate OTA artifacts	Drag-and-drop upload; server-side validation feedback; shows SHA-256, size, target device, version after upload
Manage Rollouts	Create, pause, resume, and monitor rollouts	Create rollout: select artifact → select device group → set initial percentage; increase percentage in steps; pause/resume at any time
Monitor Fleet	Real-time device fleet status dashboard	Shows: total devices, devices on each version, update success/failure counts, active rollouts with progress bars; auto-refresh every 30s

4.6 Infrastructure

Component	Technology	Purpose
Database	PostgreSQL 16	Device registry, artifacts, rollouts, telemetry events
Containerization	vasic-digital/containers submodule	Docker Compose for dev;

Component	Technology	Purpose
		Kubernetes manifests for prod
Artifact Storage	vasic-digital/storage submodule (S3-compatible)	OTA zip storage with multipart upload
Observability	vasic-digital/observability submodule	Prometheus metrics, Grafana dashboards, structured logging
Authentication	vasic-digital/auth submodule	JWT issuance, token rotation, RBAC enforcement
Caching	vasic-digital/cache submodule (Redis)	Device check-in deduplication, rollout percentage cache
Rate Limiting	vasic-digital/ratelimiter submodule	Per-device rate limits on update check (max 1/min)
Concurrency	vasic-digital/concurrency submodule	Goroutine pool for artifact validation, rollout computation
Recovery	vasic-digital/recovery submodule	Panic recovery middleware, graceful shutdown
Configuration	vasic-digital/config submodule	Environment-based config, hot-reload for non-critical params
Event Bus	vasic-digital/EventBus submodule	Async event processing for telemetry ingestion, rollout triggers
Middleware		Request logging, auth,

Component	Technology	Purpose
	vasic-digital/ middleware submodule	CORS, request ID propagation
Security	vasic-digital/ security submodule	TLS termination, certificate management, input sanitization

4.7 Core API Endpoints

```
// Device Registration
POST /api/v1/devices
// Request:
{
  "serial": "RK3588-OP5M-001",
  "model": "rk3588_opi5max",
  "current_version": "15.0.0-20260301",
  "slot_suffix": "_a",
  "hardware_fingerprint": "sha256:abc123..."
}
// Response: 201 Created
{
  "device_id": "dev_01HXYZ...",
  "auth_token": "eyJhbGciOi...",
  "check_interval_seconds": 14400
}

// Update Check
GET /api/v1/devices/{device_id}/update-check
// Response: 200 OK (update available)
{
  "update_available": true,
  "artifact_id": "art_01HABC...",
  "version": "15.0.1-20260315",
  "download_url": "/api/v1/artifacts/art_01HABC.../
    download",
  "sha256": "e3b0c44298fc1c149afb4c8996fb924...",
  "size_bytes": 1073741824,
  "signature_url": "/api/v1/artifacts/art_01HABC.../
    signature",
  "metadata": {
    "changelog_url": "/api/v1/artifacts/art_01HABC.../
      changelog",
    "mandatory": false,
    "deadline": "2026-04-15T00:00:00Z"
  }
}
```

```

// Status Report
POST /api/v1/devices/{device_id}/status
// Request:
{
  "artifact_id": "art_01HABC...",
  "status": "APPLYING",
  "progress_percent": 67,
  "error": null,
  "timestamp": "2026-03-04T12:34:56Z"
}

// Artifact Upload
POST /api/v1/artifacts
// Multipart form: file=update.zip,
  metadata={"version":"15.0.1-20260315","target_model":"rk3588_opi5max"}
// Response: 201 Created
{
  "artifact_id": "art_01HABC...",
  "sha256": "e3b0c44298fc1c149afbf4c8996fb924...",
  "size_bytes": 1073741824,
  "validation_status": "VALID",
  "created_at": "2026-03-04T10:00:00Z"
}

// Rollout Management
POST /api/v1/rollouts // Create rollout
GET /api/v1/rollouts/{id} // Get rollout
  status
PATCH /api/v1/rollouts/{id} // Update rollout (change %,
  pause/resume)
// Create Request:
{
  "artifact_id": "art_01HABC...",
  "device_group": "all",
  "initial_percentage": 5,
  "increment_step": 10,
  "auto_advance": true
}

```

4.8 Dependencies

- **No previous version dependency** (this is the first release)
- External dependencies: Android 15 AOSP update_engine, RK3588 A/B partition layout, PostgreSQL 16

4.9 New Submodules Needed

None. All infrastructure is provided by existing vasic-digital submodules.

4.10 Risk Assessment

Risk	Probability	Impact	Mitigation
update_engine API incompatibility on RK3588	Medium	Critical	Validate update_engine Binder interface on Orange Pi 5 Max in week 1; maintain compatibility shim
A/B partition layout differs from reference	Medium	High	Document exact RK3588 partition table early; test with flash_image on real hardware
Artifact upload timeout for large OTA zips (>2GB)	Low	Medium	Use chunked upload with vasic-digital/storage multipart; server-side async validation
PostgreSQL connection pool exhaustion under load	Low	Medium	vasic-digital/database provides pool configuration; load test at 10K concurrent devices
Client clock drift causes update check staleness	Low	Low	Server returns check_interval_seconds; client uses elapsed-realtime not wall clock

4.11 Testing Requirements (HelixConstitution §1.1)

Test Category	Minimum Coverage	Tool	Notes
Unit Tests	85% line coverage	Go testing, AndroidX Test	Every handler, service, and model
Mutation Testing	85% mutation score	go-mutesting (server), Stryker.NET-	Mandatory gate per §1.1

Test Category	Minimum Coverage	Tool	Notes
Integration Tests	All API endpoints	equivalent for Android testcontainers-go (PostgreSQL), Android Instrumentation	Full request/response lifecycle
End-to-End Tests	1 full OTA cycle on real hardware	Custom test harness on Orange Pi 5 Max	Upload artifact → create rollout → device checks → downloads → verifies → applies → reboots → reports success
Chaos Tests	Network partition, server restart	Custom: kill server mid-download, kill update_engine mid-apply	Device must resume or recover safely
Security Tests	Auth bypass, signature forgery, replay attacks	OWASP ZAP, custom pen-test scripts	Zero tolerance for auth bypass

5. 1.0.1-rollback — Rollback Support

5.1 Overview

Adds multi-version rollback capability, allowing administrators and devices to safely revert to a previously known-good system image. The A/B partition scheme on RK3588 already provides single-version rollback (one slot back); this version extends that to N versions.

5.2 Feature List

Feature	Description	Acceptance Criteria
Multi-Version Image Store	Keep last N system images in server-side artifact repository	N is configurable (default: 3); images are pruned on LRU basis; disk usage alerting at 80% capacity
Server-Side Rollback Trigger	Admin triggers rollback for device group or individual device	POST /api/v1/rollouts/{id}/rollback; server creates new rollout targeting the previous artifact; devices check in and receive rollback directive
Client-Initiated Rollback API	Device requests rollback via POST /api/v1/devices/{id}/rollback	Client sends current_version + target_version; server validates target is in allowed rollback window; returns rollback artifact URL
Rollback Safety Verification	Pre-rollback health check before committing rollback	Client verifies: (1) target image signature still valid, (2) battery > 30% or AC power, (3) storage sufficient for A/B swap; aborts if any check fails
Rollback Audit Trail	Every rollback is logged with reason, initiator, and outcome	Rollback events stored in PostgreSQL; dashboard shows rollback history per device; alerting on rollback rate > 5% of fleet

5.3 Key API Additions

```
// Server-side rollback trigger
POST /api/v1/rollouts/{id}/rollback
{
  "target_version": "15.0.0-20260301",
  "reason": "boot_failure_detected",
  "scope": "device_group", // or "single_device"
  "device_group": "rk3588_opi5max"
}

// Client-initiated rollback
POST /api/v1/devices/{device_id}/rollback
{
```

```

    "current_version": "15.0.1-20260315",
    "target_version": "15.0.0-20260301",
    "reason": "post_update_anomaly"
}
// Response: 200 OK
{
    "rollback_artifact_id": "art_01HDEF...",
    "download_url": "/api/v1/artifacts/art_01HDEF.../
        download",
    "sha256": "abc123...",
    "requires_reboot": true
}

```

5.4 Dependencies

Dependency	Version	Reason
1.0.0-MVP	Required	Rollback builds on MVP's artifact storage, device registry, and update pipeline

5.5 New Submodules Needed

None. Leverages existing vasic-digital/storage for image retention and vasic-digital/EventBus for rollback event propagation.

5.6 Risk Assessment

Risk	Probability	Impact	Mitigation
Rollback image no longer compatible with current bootloader	Low	Critical	Store bootloader version alongside artifact; validate bootloader compatibility before rollback
Storage exhaustion from retaining N images	Medium	High	LRU eviction with disk usage monitoring; configurable N per device group
Rollback loop	Medium	High	Circuit breaker: max 3 rollback

Risk	Probability	Impact	Mitigation
(rollback → fail → rollback → fail)			attempts per device per 24h; escalate to manual intervention
Race condition: rollback triggered while update is applying	Low	Critical	State machine lock: device can only process one update/rollback at a time; reject rollback if status=APPLYING

5.7 Testing Requirements

Test Category	Specific Tests
Unit	Rollout rollback handler, image store eviction, safety verification logic
Mutation	85% mutation score on all new rollback code paths
Integration	Server-side rollback → device receives directive → applies → reports; client- initiated rollback flow
E2E	Full rollback cycle on real hardware: update → boot failure → rollback → verify old system boots
Chaos	Kill server during rollback download; kill update_engine during rollback apply

6. 1.0.2-delta-updates — Delta/ Differential Updates

6.1 Overview

Implements delta (differential) update generation and delivery, reducing OTA payload sizes by 60-90% compared to full updates. Delta payloads contain only the binary differences between the source and target partition images.

6.2 Feature List

Feature	Description	Acceptance Criteria
Delta Payload Generation	Server-side service generates delta payloads between consecutive versions	Uses brillo_update_payload or bsdiff algorithm; generates delta within 4 hours for 2GB partitions; stores delta alongside full artifact
Delta Generation Service	Background worker that generates deltas upon artifact upload	Triggered by EventBus on artifact.uploaded event; queues delta generation; reports generation status via API
Bandwidth Reduction	Delta payloads are 60-90% smaller than full payloads	Measured on real RK3588 images: minor version bump $\geq$ 60% reduction; patch-level bump $\geq$ 80% reduction
Automatic Delta Selection	Server selects delta vs. full update based on device's current version	Device reports source_version in update check; server returns delta if available for source→target; falls back to full update otherwise
Delta Fallback	Client falls back to full update if delta apply fails	update_engine returns DELTA_APPLICATION_FAILED; client re-requests update check with delta_failed=true; server returns full payload
Delta Integrity Verification	Delta payloads are verified same as full payloads	SHA-256 + RSA signature on delta payload; source partition hash verified before delta apply

6.3 Key API Additions

```
// Update check with delta support
GET /api/v1/devices/{device_id}/update-check?
    source_version=15.0.0-20260301
// Response: 200 OK (delta available)
{
  "update_available": true,
  "artifact_id": "art_delta_01H...",
  "update_type": "DELTA",
  "source_version": "15.0.0-20260301",
```

```

    "target_version": "15.0.1-20260315",
    "download_url": "/api/v1/artifacts/art_delta_01H.../
        download",
    "sha256": "def456...",
    "size_bytes": 214748364, // ~200MB vs ~1GB full
    "size_savings_percent": 80,
    "full_update_fallback_url": "/api/v1/artifacts/
        art_01HABC.../download"
}

// Delta generation status
GET /api/v1/artifacts/{source_id}/deltas
// Response:
{
  "deltas": [
    {
      "target_artifact_id": "art_01HABC...",
      "status": "GENERATED",
      "size_bytes": 214748364,
      "size_savings_percent": 80,
      "generation_time_seconds": 7200
    }
  ]
}

```

6.4 Dependencies

Dependency	Version	Reason
1.0.0-MVP	Required	Delta builds on MVP's artifact storage and update pipeline
1.0.1-rollback	Recommended	Delta updates increase rollback complexity; having rollback support first reduces risk

6.5 New Submodules Needed

Submodule	Purpose
helix-delta-gen	Delta generation service; wraps brillo_update_payload /

Submodule	Purpose
	bsdiff; runs as background worker

6.6 Risk Assessment

Risk	Probability	Impact	Mitigation
Delta generation takes too long (>8h for large partitions)	Medium	Medium	Parallelize by partition; incremental delta (only changed partitions); limit delta to consecutive versions
Delta apply fails on device due to source partition drift	Medium	High	Verify source partition SHA-256 before delta apply; fall back to full update on mismatch
Storage overhead: $N \times M$ delta combinations (N source $\times$ M target)	Medium	Medium	Only generate deltas for consecutive versions; limit to last 3 source versions
bsdiff memory usage during generation (>16GB RAM for 2GB partition)	Low	High	Use streaming bsdiff variant; run on high-memory builder node; cap memory at 32GB

6.7 Testing Requirements

Test Category	Specific Tests
Unit	Delta selection logic, source→target mapping, fallback trigger
Mutation	

Test Category	Specific Tests
	85% mutation score on delta generation and selection code
Integration	Upload artifact → delta generated → device checks in → receives delta → applies
E2E	Full delta update cycle on real hardware; measure actual bandwidth reduction
Performance	Delta generation time benchmarked at $\leq 4h$ for 2GB partition

7. 1.1.0-linux-support — Linux Distribution OTA

7.1 Overview

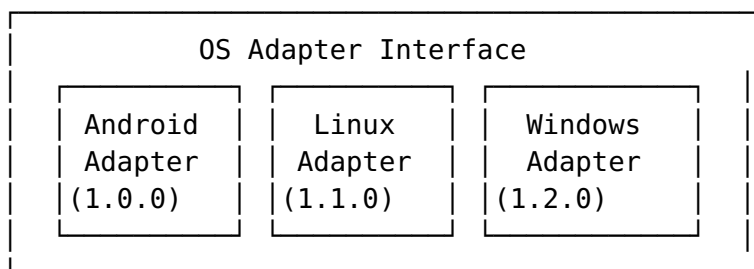
Extends Helix OTA to support Linux distribution updates on RK3588 and generic x86_64/ARM64 hardware. This is a **minor version bump** (1.1.0) because it adds a new OS platform without changing the existing Android pipeline.

7.2 Feature List

Feature	Description	Acceptance Criteria
Ubuntu/Debian APT Updates	Trigger apt upgrade via OTA; distribute custom .deb packages	Server pushes package list; client runs apt-get upgrade --assume-yes; reports success/failure per package; supports dry-run mode
Fedora/rpm-ostree Atomic Updates	Integrate with rpm-ostree for atomic system updates	Client runs rpm-ostree upgrade; transactional: either all packages apply or none; instant rollback via rpm-ostree rollback
Arch Linux pacman Updates	Trigger pacman -Syu via OTA with custom repository support	Client syncs custom repository; applies updates; handles .pacnew file

Feature	Description	Acceptance Criteria
		merging; reports conflicts
Generic Linux Rootfs A/B Updates	Full rootfs A/B swap for any Linux distribution	Client writes new rootfs to inactive partition; updates bootloader; reboots; identical flow to Android A/B but for Linux rootfs
OSTree Integration	OSTree-based update delivery for immutable Linux distros	Client pulls OSTree commit from Helix server (acting as OSTree remote); deploys new commit; atomic swap

7.3 Key Architecture Addition



Each OS adapter implements a common interface:

```

// OSAdapter defines the contract for OS-specific update logic
type OSAdapter interface {
    // CheckForUpdate queries the device's current state and returns
    // whether an update is available for this OS type.
    CheckForUpdate(ctx context.Context, device Device)
        (*UpdateInfo, error)

    // ApplyUpdate triggers the OS-specific update mechanism.
    ApplyUpdate(ctx context.Context, device Device, artifact Artifact) error

    // VerifyUpdate confirms the update was applied correctly.
    VerifyUpdate(ctx context.Context, device Device) error

    // Rollback reverts to the previous system state.
    Rollback(ctx context.Context, device Device) error

    // ReportStatus returns the current update status.
  
```

```

        ReportStatus(ctx context.Context, device Device)
            (*UpdateStatus, error)
    }

```

7.4 Dependencies

Dependency	Version	Reason
1.0.0-MVP	Required	Server infrastructure, dashboard, device registry
1.0.1-rollback	Required	Rollback is critical for Linux A/B updates and rpm-ostree
1.0.2-delta-updates	Recommended	Delta updates equally valuable for Linux rootfs updates

7.5 New Submodules Needed

Submodule	Purpose
helix-linux-client	Linux OTA client (Go binary); supports APT, rpm-ostree, pacman, rootfs A/B, OSTree
helix-ostree-adapter	Server-side OSTree repository management and delta generation

7.6 Risk Assessment

Risk	Probability	Impact	Mitigation
APT dependency hell on heterogeneous device fleet	High	High	Pin package versions server-side; pre-validate dependency trees in CI before rollout
rpm-ostree layering conflicts	Medium	Medium	Document supported layering

Risk	Probability	Impact	Mitigation
pacman .pacnew handling differs across installations	Medium	Low	patterns; reject updates with conflicting layers Default: preserve existing config; report .pacnew files for manual review
Rootfs A/B requires bootloader-specific integration (GRUB, U-Boot, systemd-boot)	High	Critical	Support U-Boot first (RK3588 native); add GRUB and systemd-boot adapters incrementally
OSTree static delta generation adds server complexity	Medium	Medium	Reuse helix-delta-gen architecture; extend for OSTree delta format

7.7 Testing Requirements

Test Category	Specific Tests
Unit	Each OS adapter's CheckForUpdate, ApplyUpdate, Rollback implementations
Mutation	85% mutation score on all OS adapter code
Integration	Each Linux adapter against containerized OS images (Ubuntu, Fedora, Arch)
E2E	Full A/B rootfs update on RK3588 running Debian; rpm-ostree upgrade on Fedora VM; pacman update on Arch container
Chaos	Kill apt/pacman mid-transaction; verify rollback

Test Category	Specific Tests
	works; test with broken package repositories

8. 1.2.0-windows-support — Windows OTA

8.1 Overview

Adds Windows as a supported OTA target platform, enabling centralized update management for Windows devices alongside Android and Linux fleets.

8.2 Feature List

Feature	Description	Acceptance Criteria
Windows Update Wrapper	Integrate with Windows Update API to manage OS updates via Helix	Client calls Windows Update Agent API; Helix server controls approval timing; reports WSUS-like compliance data
MSI/MSIX Package Distribution	Distribute custom MSI/MSIX packages via OTA	Server stores MSI/MSIX artifacts; client downloads and installs via <code>msiexec / Add-AppxPackage</code> ; rollback via MSI uninstall
Windows Service Client	Helix OTA client runs as a Windows Service	Installed as NT service with auto-start; runs under <code>LOCAL_SYSTEM</code> ; communicates with server over HTTPS; supports service pause/resume
Registry-Based Configuration	Store client configuration in Windows Registry	Registry path: <code>HKLM\SOFTWARE\HelixOTA\Client</code> ; configurable: server URL, check interval, proxy settings; changes take effect without restart

8.3 Windows Client Architecture

```
// Windows Service main loop
func (s *HelixOTAService) Execute() error {
    ticker := time.NewTicker(s.config.CheckInterval)
    for {
        select {
        case <-ticker.C:
            update, err := s.checkForUpdate()
            if err != nil {
                s.log.Error("update check failed", "error",
err)
                continue
            }
            if update != nil {
                s.applyUpdate(update)
            }
        case <-s.shutdownCh:
            return nil
        }
    }
}
```

8.4 Dependencies

Dependency	Version	Reason
1.1.0-linux-support	Required	OS adapter interface defined in 1.1.0; Windows adapter implements the same interface

8.5 New Submodules Needed

Submodule	Purpose
helix-windows-client	Windows Service client (Go binary cross-compiled for windows/amd64)
helix-msi-packager	MSI packaging tool for creating Helix OTA installer

8.6 Risk Assessment

Risk	Probability	Impact	Mitigation
Windows Update API requires elevated privileges	High	Medium	Run service as LOCAL_SYSTEM; document privilege requirements
MSI rollback on partial install failure	Medium	High	Use MSI transaction support; test all rollback paths
Windows Defender SmartScreen blocks client binary	Medium	Medium	Code-sign with EV certificate; submit to Microsoft for whitelisting
Cross-compilation from Linux produces subtle runtime issues	Medium	High	Run full test suite on native Windows; use Windows CI runner for validation
Registry permission issues on locked-down corporate devices	Low	High	Document required registry permissions; provide Group Policy template

8.7 Testing Requirements

Test Category	Specific Tests
Unit	Windows adapter, MSI packager, registry configuration reader
Mutation	85% mutation score on Windows adapter code
Integration	Windows Service lifecycle (install → start → check → update → stop → uninstall)
E2E	Full MSI distribution cycle on Windows 10/11 VM;

Test Category	Specific Tests
Security	Windows Update integration test
	Verify service runs with minimal privileges; test with UAC enabled; verify TLS certificate validation

9. 2.0.0-multi-os-universal — Universal Multi-OS

9.1 Overview

The 2.0.0 release unifies all OS-specific adapters into a **plugin architecture**, making Helix OTA a truly OS-agnostic update platform. This is a **major version bump** because it introduces a breaking change to the internal adapter API (though external APIs remain backward-compatible).

9.2 Feature List

Feature	Description	Acceptance Criteria
Plugin Architecture	OS adapters are dynamically loaded plugins (Go plugins or WASM)	New OS support added without recompiling server; plugins declare capabilities via manifest; server validates plugin signatures
OS-Agnostic Update Pipeline	Unified pipeline that works for any OS type	Pipeline stages: check → download → verify → apply → report; each stage delegates to plugin; stages are composable
Unified Dashboard	Single dashboard for all OS types	Dashboard shows devices across Android, Linux, Windows in one view; filters by OS type; OS-specific detail views
Cross-OS Dependency Management	Track and manage dependencies	E.g., Android app update requires corresponding backend API version;

Feature	Description	Acceptance Criteria
Plugin SDK	that span OS boundaries	dependency graph validated before rollout
	Public SDK for building third-party OS adapters	SDK includes: adapter interface, test harness, documentation, example plugin (FreeBSD); plugins compile against stable API

9.3 Plugin Architecture

```
// Plugin manifest (plugin.yaml)
```

```
name: helix-adapter-freebsd
```

```
version: 1.0.0
```

```
os_type: freebsd
```

```
capabilities:
```

- full_update
- delta_update
- rollback

```
signature: "RSA-SHA256:abc123..."
```

```
// Plugin loading
```

```
func LoadPlugin(ctx context.Context, path string)
```

```
(OSAdapter, error) {
```

```
    manifest, err := ParseManifest(path + "/plugin.yaml")
```

```
    if err != nil {
```

```
        return nil, fmt.Errorf("parse manifest: %w", err)
```

```
    }
```

```
    if err := VerifyPluginSignature(manifest); err != nil {
```

```
        return nil, fmt.Errorf("signature verification failed: %w", err)
```

```
    }
```

```
    adapter, err := pluginOpen(path + "/adapter.so")
```

```
    if err != nil {
```

```
        return nil, fmt.Errorf("load plugin: %w", err)
```

```
    }
```

```
    return adapter, nil
```

```
}
```

9.4 Dependencies

Dependency	Version	Reason
1.0.0-MVP through 1.2.0-windows	All required	Plugin architecture extracts and generalizes all

Dependency	Version	Reason
1.0.1-rollback	Required	existing adapters Rollback must work across all OS plugins
1.0.2-delta-updates	Required	Delta generation must be plugin-aware

9.5 New Submodules Needed

Submodule	Purpose
helix-plugin-sdk	Public SDK for building OS adapter plugins
helix-plugin-registry	Plugin storage, versioning, and distribution service
helix-adapter-freebsd	Reference third-party plugin (FreeBSD support)

9.6 Risk Assessment

Risk	Probability	Impact	Mitigation
Go plugin system has compatibility limitations (compiler version, libc)	High	Critical	Evaluate WASM as alternative plugin runtime; provide WASM SDK as fallback
Plugin sandboxing: malicious plugin could compromise server	Medium	Critical	Run plugins in isolated process; restrict filesystem/network access; sign all plugins
Breaking change to adapter API alienates	Medium	High	Maintain backward-compatible API wrapper;

Risk	Probability	Impact	Mitigation
existing users			deprecation period of 2 minor versions
Cross-OS dependency graph becomes intractable	Medium	Medium	Limit dependency depth to 3 levels; provide visualization tool; warn on circular dependencies
Plugin load-time performance regression	Low	Medium	Lazy-load plugins; cache compiled plugin instances; benchmark $\leq 100\text{ms}$ plugin load time

9.7 Testing Requirements

Test Category	Specific Tests
Unit	Plugin loader, manifest parser, signature verifier, dependency graph resolver
Mutation	85% mutation score on plugin infrastructure code
Integration	Load/unload plugin at runtime; plugin communicates with server pipeline; hot-swap plugin version
E2E	Full update cycle using third-party plugin (FreeBSD); verify plugin isolation
Security	Pen-test plugin sandbox; attempt privilege escalation from plugin; verify signature rejection for tampered plugins
Performance	Plugin load time $\leq 100\text{ms}$; plugin call overhead $\leq 1\text{ms}$ per stage

10. Cross-Cutting Concerns

10.1 Security Model (All Versions)

Concern	Implementation
Transport Security	TLS 1.3 mandatory; certificate pinning on client
Payload Integrity	SHA-256 hash verified before apply; hash embedded in signed metadata
Payload Authenticity	RSA-2048/4096 signature on all artifacts; public key embedded in client
Device Authentication	mTLS or JWT with per-device secrets; token rotation every 24h
Server Authentication	Client validates server certificate against pinned CA
Supply Chain	Artifact signing occurs in isolated CI pipeline; signing key never leaves HSM
Replay Protection	Nonce-based request signing; server rejects replayed update checks

10.2 Observability (All Versions)

Every version must expose:

- **Metrics:** Prometheus-compatible metrics for request latency, error rates, device check-in frequency, update success/failure rates, artifact download throughput
- **Logging:** Structured JSON logging with request IDs; log levels: DEBUG, INFO, WARN, ERROR; PII redaction
- **Tracing:** OpenTelemetry distributed tracing from dashboard → server → client (where network allows)
- **Alerting:** PagerDuty integration for: update failure rate > 5%, artifact validation failure, server error rate > 1%, database connection pool exhaustion

10.3 Backward Compatibility

From → To	API Compatibility	Client Compatibility	Database Migration
1.0.0 → 1.0.1	Additive only	1.0.0 clients work with 1.0.1 server	Forward-compatible

From → To	API Compatibility	Client Compatibility	Database Migration
1.0.1 → 1.0.2	Additive only	1.0.1 clients work with 1.0.2 server	Forward- compatible
1.0.2 → 1.1.0	Additive only	1.0.x Android clients work with 1.1.0 server	Forward- compatible
1.1.0 → 1.2.0	Additive only	1.1.x clients work with 1.2.0 server	Forward- compatible
1.2.0 → 2.0.0	Additive external; internal adapter API changes	1.x clients work with 2.0.0 server (external API compatible)	Migration script provided

Appendix A — Submodule Dependency Map

```

vasic-digital/
├─ auth          ← Used by: 1.0.0+ (device auth,
dashboard auth)
├─ database      ← Used by: 1.0.0+ (PostgreSQL access
layer)
├─ cache         ← Used by: 1.0.0+ (Redis for check-in
dedup, rollout cache)
├─ observability ← Used by: 1.0.0+ (metrics, logging,
tracing)
├─ security      ← Used by: 1.0.0+ (TLS, cert management,
input sanitization)
├─ middleware    ← Used by: 1.0.0+ (request logging,
auth, CORS)
├─ config        ← Used by: 1.0.0+ (environment config,
hot-reload)
├─ EventBus      ← Used by: 1.0.0+ (async telemetry,
rollout triggers)
├─ Storage       ← Used by: 1.0.0+ (artifact storage, S3-
compatible)
├─ ratelimiter   ← Used by: 1.0.0+ (per-device rate
limiting)
├─ concurrency   ← Used by: 1.0.0+ (goroutine pools for
validation)
├─ recovery      ← Used by: 1.0.0+ (panic recovery,
graceful shutdown)

```


Appendix B — Testing Standards Reference

Per HelixConstitution §1.1, the following testing standards apply to **all versions**:

B.1 Mutation Testing Requirements

1. **Scope:** All production Go code in the server; all critical-path code in Android/Linux/Windows clients
2. **Minimum Score:** 85% mutation kill rate
3. **Tooling:** go-mutesting for server; platform-native mutation tools for clients
4. **Gate:** CI pipeline blocks merge if mutation score drops below 85%
5. **Frequency:** Mutation testing runs on every PR targeting a release branch

B.2 Test Coverage Requirements

Code Category	Minimum Line Coverage	Minimum Branch Coverage
API Handlers	90%	85%
Business Logic (Rollout, Artifact)	95%	90%
Data Models	80%	75%
Client Update Pipeline	90%	85%
OS Adapters	85%	80%
Plugin Infrastructure (2.0.0)	90%	85%

B.3 Test Environment Requirements

Environment	Purpose	Provisioning
Local (Docker Compose)	Developer testing	docker compose up from vasic-digital/containers

Environment	Purpose	Provisioning
CI (GitHub Actions / GitLab CI)	Automated testing on PR	Testcontainers for PostgreSQL, Redis
Staging (Kubernetes)	Pre-release validation	basic-digital/containers K8s manifests
Hardware Lab	Orange Pi 5 Max real-device testing	Minimum 2 devices per test run
Chaos Lab	Network partition, power failure testing	Custom harness: tc netem, iptables, controlled power cuts

End of Document — Helix OTA Version Roadmap v1.0.0-draft
This document is a living artifact. Updates require review per HelixConstitution §3.2 (nano-detail documentation standard).